[승코의 차근차근 Flutter] 11, 싱글톤 패턴과 Factory

Dart를 공부하다 보면 독특한 키워드 중 하나가 바로 `factory` 입니다. 언뜻 보면 그냥 생성자처럼 보이지만, 내부적으로는 매우 강력한 기능을 제공합니다. 이번 글에서는 `factory` 키워드의 역할과 함께, 이를 활용해 대표적인 디자인 패턴인 **싱글톤(Singleton)**을 구현하는 방법까지 깊이 있게 알아보겠습니다.

✅ 1. factory란 무엇인가?

Dart의 `factory` 키워드는 **객체 생성 방식에 유연성을 부여**하는 생성자입니다. 일반적인 생성자(`constructor`)는 항상 새로운 객체를 반환하지만, `factory`는 **이미 생성된 객체를 반환**하거나, **조건에 따라 다른 클래스를 반환**할 수도 있습니다. 즉, 객체 생성 과정을 개발자가 직접 통제할 수 있게 되는 것이죠.

✔️ 일반 생성자 vs factory 생성자

항목	일반 생성자	factory 생성자
항상 새로운 인스턴스 생성	✅	❌
조건에 따라 인스턴스 반환	❌	✅
하위 클래스 반환 가능	❌	✅
this 접근 가능	✅	❌ (객체가 아직 생성 전)

🔍 2. factory를 사용하는 주요 이유

1) 싱글톤 패턴 구현

싱글톤 패턴(Singleton Pattern)은 **앱 전역에서 단 하나의 인스턴스만 존재하도록 보장하는 디자인 패턴**입니다. 이를 통해 자원 낭비를 방지하고, 여러 곳에서 동일한 상태를 공유할 수 있게 됩니다. Dart에서 싱글톤을 구현할 때 `factory`를 사용하면 간결하고 명확하게 작성할 수 있습니다.

2) 하위 클래스 반환 (다형성)

`factory` 생성자를 이용하면 부모 클래스의 생성자가 하위 클래스의 인스턴스를 반환할 수 있어, **팩토리 메서드 패턴**이나 **전략 패턴** 구현에 유리합니다.

3) 외부 데이터 기반 객체 생성

JSON, XML 등의 외부 데이터를 객체로 변환할 때도 `factory` 생성자가 자주 사용됩니다. 복잡한 로직을 캡슐화하여 읽기 쉽고 유지보수하기 좋은 코드를 만들 수 있습니다.

🧪 3. 코드로 배우는 factory 생성자

📌 예제 1: JSON 파싱용 생성자

```
class Book {
  final String title;
  final String description;

  Book({required this.title, required this.description});

  factory Book.fromJson(Map<String, dynamic> json) {
    return Book(
      title: json['title'],
      description: json['description'],
    );
  }
}
```

- 외부 JSON 데이터를 받아 내부 객체로 바꿔주는 경우.
- `fromJson()` 은 객체 생성을 통제해야 하므로 `factory` 사용.

📌 예제 2: 싱글톤 패턴 구현 (전형적 형태)

```
class AppConfig {
  static final AppConfig _instance = AppConfig._internal();

  factory AppConfig() {
```

```
    return _instance;
  }

  AppConfig._internal();

  // 예: 설정값
  String baseUrl = "https://api.example.com";
}
```

- `AppConfig()`를 아무리 호출해도 항상 동일한 `_instance` 반환.
- 생성자는 비공개(`_internal`)로 막아 외부에서 직접 인스턴스 생성 불가.
- 이 방식은 간단한 싱글톤 구현에 적합합니다.

📌 예제 3: 조건 기반 싱글톤 (이름별로 하나씩 유지)

```
class Logger {
  final String name;
  bool mute = false;

  static final Map<String, Logger> _cache = <String, Logger>{};

  factory Logger(String name) {
    return _cache.putIfAbsent(name, () => Logger._internal(name));
  }

  Logger._internal(this.name);

  void log(String msg) {
    if (!mute) print("[$name] $msg");
  }
}
```

- 로그 인스턴스를 이름별로 하나씩만 생성.
- `_cache`를 통해 동일한 이름에 대해 같은 객체 반환.

- 멀티 인스턴스를 제한적으로 허용하는 응용형 싱글톤.

📌 예제 4: 하위 클래스 반환

```
abstract class Book {
  factory Book(String type, String title) {
    switch (type) {
      case 'comic':
        return ComicBook(title);
      case 'novel':
        return NovelBook(title);
      default:
        throw ArgumentError('Unknown book type');
    }
  }
}

class ComicBook implements Book {
  final String title;
  ComicBook(this.title);
}

class NovelBook implements Book {
  final String title;
  NovelBook(this.title);
}
```

- `Book` 클래스는 추상 클래스이며, 실제 생성되는 인스턴스는 `ComicBook`, `NovelBook` 등 하위 클래스.
- 조건에 따라 다양한 하위 클래스 반환 가능 → 다형성과 캡슐화 실현.

⚠️ factory 사용 시 주의사항

- `factory` 내부에서는 `this`를 사용할 수 없습니다. 객체가 아직 완전히 생성되지 않았기 때문입니다.

- 일반 생성자처럼 `super`를 호출하거나 생성자 초기화 리스트를 사용할 수 없습니다.
- 인스턴스를 직접 반환하므로, 어떤 객체가 반환되는지 명확하게 문서화하거나 네이밍으로 표현하는 것이 중요합니다.

🎯 언제 factory 생성자를 사용해야 할까?

상황	factory 사용 권장 여부
싱글톤 패턴 (인스턴스 재사용)	✅
JSON, XML 등 외부 데이터 파싱	✅
다형성 구현 (하위 클래스 반환)	✅
무조건 새로운 객체가 필요한 경우	❌

🧠 마무리 정리

- Dart의 `factory` 키워드는 단순한 문법이 아닌, **객체 생성에 대한 유연성과 제어력을 제공하는 강력한 도구**입니다.
- 특히 **싱글톤 구현**, **파싱 로직 분리**, **다형성 처리**에 탁월한 기능을 제공합니다.
- 디자인 패턴과 함께 이해하면 코드의 확장성과 유지보수성이 크게 향상됩니다.

> Dart를 더 깊이 이해하고 싶다면, factory 생성자의 다양한 활용 예를 직접 구현해보세요. 실전에서 빛을 발하는 개념입니다.

C.P.
COMPANY